



Unstuck Guide

bank_api Extension 6 · Authentication & Authorization lab

Hints first, code last. Every task has three hint levels, then a reference solution and a way to check it.

READ ME FIRST

How to use this deck



1. Run the failing test

Run the task's test file with `-v -x`.
Read the assertion: it tells you exactly what is expected.



2. Read ONE hint level

Nudge -> Pointer -> Almost there. Try each for 5 minutes before the next.



3. Reference code last

Understand every line, then type it yourself. Copy-paste teaches nothing.



4. Verify

Rerun the tests for that task, then the whole suite so nothing else broke.

Most useful pytest flags `-x` stop at first failure `-k name` run one test `--lf` rerun last failures `-q` quiet summary

SETUP

Stuck before Task 1? Setup problems

You see	Cause	Fix
<code>ModuleNotFoundError: No module named 'jwt'</code>	PyJWT not installed	<code>pip install PyJWT</code> (NOT " <code>pip install jwt</code> ": different package, same import name)
<code>Form data requires "python-multipart"</code>	Missing dependency of OAuth2PasswordRequestForm	<code>pip install python-multipart</code>
<code>ValidationError ... JWT_SECRET_KEY</code>	Secret missing or < 32 chars	Check <code>.env</code> ; start uvicorn from the <code>banking_api</code> folder so <code>.env</code> is found
<code>no such column: accounts.owner_id</code>	Old bank.db from Extension 5	Stop server, delete bank.db, restart (<code>create_all</code> builds the new schema)
<code>[seed_admin] hash_password() not implemented</code>	Task 1 not done yet	Expected. The admin is created on the first start after Task 1
<code>All Extension 5 tests ERROR in setup</code>	client fixture cannot log in yet	Expected until Tasks 1 and 2 are done. Work task by task

Task 1: Users & safe password storage



Goal

POST /api/v1/auth/register creates a CUSTOMER with an Argon2id hash. No hash in any response. Role cannot be chosen by the client.

Files

```
app/core/security.py
app/schemas/auth_schema.py
app/services/auth_service.py
```

1 NUDGE

Three layers: the SCHEMA validates input, the SERVICE applies rules (unique name, role), SECURITY hashes. Which test is failing tells you which layer is wrong.

2 POINTER

pwdlib already did the hard work: `_password_hash.hash(pw)` and `_password_hash.verify(pw, hashed)`. In Pydantic v2, reject unknown fields with `model_config = ConfigDict(extra="forbid")` and constrain strings with `Field(min_length=..., max_length=..., pattern=...)`.

3 ALMOST THERE

`register()`: `username = username.lower()`; if `self.repo.get_by_username(username)`: raise `UsernameTakenError(username)`; then `self.repo.create(User(username=..., hashed_password=hash_password(password), role=Role.CUSTOMER))`. `UserOut` needs `role`, `is_active` and `created_at` too.

Task 1: reference code

```
# security.py
def hash_password(plain: str) -> str:
    return _password_hash.hash(plain)

def verify_password(plain: str, hashed: str) -> bool:
    return _password_hash.verify(plain, hashed)

# auth_schema.py
class UserCreate(BaseModel):
    model_config = ConfigDict(extra="forbid")
    username: str = Field(min_length=3, max_length=32,
                          pattern=r"^[A-Za-z0-9_.-]+$")
    password: str = Field(min_length=15, max_length=128)

class UserOut(BaseModel):
    id: int
    username: str
    role: str
    is_active: bool
    created_at: datetime
    model_config = ConfigDict(from_attributes=True)

# auth_service.py
def register(self, username: str, password: str) -> User:
    username = username.lower()
    if self.repo.get_by_username(username):
        raise UsernameTakenError(username)
    return self.repo.create(User(username=username,
                                  hashed_password=hash_password(password), role=Role.CUSTOMER))
```



Common mistakes

- Returning the ORM User with a schema that lists hashed_password
- Checking duplicates before lower-casing ("Asha" vs "asha")
- Letting the client send role and trusting it
- Using hashlib.sha256 "because it is a hash"



Verify

```
pytest tests/test_auth1_users.py -v
-> 7 passed
```

Task 2: Login issues a JWT



Goal

POST /api/v1/auth/token (form data) returns {access_token, token_type, expires_in}. Claims: sub (string), role, iss, iat, exp. One generic 401 for every failure.

Files

```
app/core/security.py
app/services/auth_service.py
app/routers/auth_router.py
```

1 NUDGE

A JWT is just a dict signed with a secret. Decide the claims, pick the expiry, sign. For login, think like an attacker: what could the response message or the response TIME reveal?

2 POINTER

jwt.encode(claims, settings.JWT_SECRET_KEY, algorithm=settings.JWT_ALGORITHM). PyJWT accepts datetime objects for iat/exp. The route takes form: OAuth2PasswordRequestForm = Depends(), which gives you form.username and form.password.

3 ALMOST THERE

In login(): ok = verify_password(password, user.hashed_password if user else DUMMY_HASH), THEN if not user or not ok or not user.is_active: raise NotAuthenticatedError("Incorrect username or password"). "sub" must be str(user_id).

Task 2: reference code

```
# security.py
DUMMY_HASH = hash_password("dummy-password-for-timing-safety")

def create_access_token(user_id: int, role: str) -> str:
    now = datetime.now(timezone.utc)
    ttl = timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    claims = {"sub": str(user_id), "role": role,
              "iss": settings.JWT_ISSUER, "iat": now, "exp": now + ttl}
    return jwt.encode(claims, settings.JWT_SECRET_KEY,
                      algorithm=settings.JWT_ALGORITHM)

# auth_service.py
def login(self, username: str, password: str) -> dict:
    user = self.repo.get_by_username(username)
    ok = verify_password(password,
                          user.hashed_password if user else DUMMY_HASH)
    if not user or not ok or not user.is_active:
        raise NotAuthenticatedError("Incorrect username or password")
    return {"access_token": create_access_token(user.id, user.role),
            "token_type": "bearer",
            "expires_in": settings.ACCESS_TOKEN_EXPIRE_MINUTES * 60}

# auth_router.py
@router.post("/token", response_model=Token)
def login(form: OAuth2PasswordRequestForm = Depends(),
          service: AuthService = Depends()):
    return service.login(form.username, form.password)
```



Common mistakes

- "sub": user.id (int) -> InvalidSubjectError on decode
- datetime.now() without timezone.utc
- Different messages for "no such user" and "wrong password"
- Sending JSON to /token in curl (needs -d "username=..&password=..")



Verify

```
pytest tests/test_auth2_login.py -v
-> 4 passed
```

TASK 3 · HINTS (TRY EACH FOR 5 MINUTES)

Task 3: Who is calling? get_current_user



Goal

A dependency that turns "Authorization: Bearer <jwt>" into a User, or raises one generic 401. Plus GET /api/v1/auth/me.

Files

```
app/core/security.py
app/core/auth.py
app/routers/auth_router.py
```

1 NUDGE

List every way a token can be bad (missing, garbage, wrong key, tampered, expired, alg none, wrong issuer, user gone). You do NOT need an if for each: PyJWT raises for most of them when you give it the right arguments.

2 POINTER

`jwt.decode(token, key, algorithms=[...], issuer=..., options={"require": [...]}).` All PyJWT failures inherit from `jwt.PyJWTError`, so one except block covers them. `int("abc")` raises `ValueError`; a missing key raises `KeyError`.

3 ALMOST THERE

`if not token: raise NotAuthenticatedError("Not authenticated").` try: `claims = decode_access_token(token); user_id = int(claims["sub"])` except (`jwt.PyJWTError`, `ValueError`, `KeyError`): `raise NotAuthenticatedError("Invalid or expired token").` Then `users.get_by_id(user_id);` None or inactive -> same 401.

Task 3: reference code

```
# security.py
def decode_access_token(token: str) -> dict:
    return jwt.decode(token, settings.JWT_SECRET_KEY,
                      algorithms=[settings.JWT_ALGORITHM],
                      issuer=settings.JWT_ISSUER,
                      options={"require": ["exp", "iat", "sub", "iss"]})

# auth.py
def get_current_user(token: Optional[str] = Depends(oauth2_scheme),
                    users: UserRepository = Depends()) -> User:
    if not token:
        raise NotAuthenticatedError("Not authenticated")
    try:
        claims = decode_access_token(token)
        user_id = int(claims["sub"])
    except (jwt.PyJWTError, ValueError, KeyError):
        raise NotAuthenticatedError("Invalid or expired token")
    user = users.get_by_id(user_id)
    if user is None or not user.is_active:
        raise NotAuthenticatedError("Invalid or expired token")
    return user

# auth_router.py
@router.get("/me", response_model=UserOut)
def me(user: User = Depends(get_current_user)):
    return user
```



Common mistakes

- algorithms=jwt.get_unverified_header(token)["alg"] (attacker picks!)
- options={"verify_signature": False} left in from debugging
- Catching Exception: hides real bugs as 401s
- Forgetting issuer= so tokens from other services pass



Verify

```
pytest tests/test_auth3_current_user.py -v
-> 9 passed
```

TASK 4 · HINTS (TRY EACH FOR 5 MINUTES)

Task 4: Lock the doors



Goal

No money endpoint (v1 or legacy) answers without a valid token. Public: /, /docs, /openapi.json, /auth/register, /auth/token.

Files

```
app/routers/account_router.py
app/routers/scheduled_payment_router.py
app/routers/transaction_router.py
```

1 NUDGE

You could add a user parameter to 14 functions... or tell the ROUTER that every route it owns needs a dependency. What happens to routes you add next month?

2 POINTER

APIRouter(prefix=..., tags=..., dependencies=[Depends(get_current_user)]). The account router is included twice (legacy and /api/v1) so one edit covers both. Auth routes live on a different router, so they stay public.

3 ALMOST THERE

Edit three routers: account_router.py and scheduled_payment_router.py at router level; transaction_router.py at router or route level. Then run the test file: the only remaining failure should be /api/v1/admin/users (fixed in Task 6).

Task 4: reference code

```
# account_router.py
from ..core.auth import get_current_user

router = APIRouter(prefix="/accounts", tags=["Accounts"],
                   dependencies=[Depends(get_current_user)])

# scheduled_payment_router.py
router = APIRouter(tags=["Scheduled Payments"],
                   dependencies=[Depends(get_current_user)])

# transaction_router.py (Task 5 replaces this with can_view_account)
@router.get("/{account_id}/transactions",
            response_model=Page[TransactionOut],
            dependencies=[Depends(get_current_user)])
def list_transactions(...): ...

# Why does the 401 carry "WWW-Authenticate: Bearer"?
# NotAuthenticatedError.headers + the Extension 5 handler now
# passes exc.headers into the JSONResponse (provided).
```



Common mistakes

- Adding the dependency to app = FastAPI(dependencies=...) and locking /auth/token too
- Using OAuth2PasswordBearer with auto_error=True -> 401 without the v1 envelope
- Protecting only /api/v1 and forgetting legacy /accounts



Verify

```
pytest tests/test_auth4_*.py -v
-> 20 passed (21 after Task 6)
```

Task 5: Is it yours? Ownership (BOLA)



Goal

Accounts belong to users. Customers can only see and use their own; others get 404. Admins can view but not move money (403).

Files

```
app/models/account.py
app/core/auth.py
app/services/account_service.py
app/repositories/account_repository.py
app/routers/*.py
```

1 NUDGE

Two decisions per request: 1) does the account exist and 2) may THIS user do THIS kind of action on it. Reading and moving money are different kinds of action. Where do you know the account id: the path, the body, or a parent object?

2 POINTER

Add `owner_id` to `Account` and set it in `create_account`. `AccountAccess` gets the user and the repository by `Depends`; `view()` allows owner or ADMIN, `operate()` allows owner only. Put `dependencies=[Depends(can_view_account)]` on routes with `{account_id}` in the path: FastAPI fills `account_id` automatically.

3 ALMOST THERE

Transfer: `access: AccountAccess = Depends()`; `access.operate(request.from_account_id)`. Payments by id: `payment = service.get(payment_id)`; `access.view(payment.account_id)`. Idempotency: `scoped_key = f"user:{access.user.id}:{idempotency_key}"`.

Task 5: reference code (core)

```
class AccountAccess:
    def __init__(self, user: User = Depends(get_current_user),
                 accounts: AccountRepository = Depends()):
        self.user, self.accounts = user, accounts

    def _load(self, account_id: int) -> Account:
        account = self.accounts.get_by_id(account_id)
        if account is None:
            raise AccountNotFoundError(account_id)
        return account

    def view(self, account_id: int) -> Account:
        account = self._load(account_id)
        if account.owner_id == self.user.id or self.user.role == Role.ADMIN:
            return account
        raise AccountNotFoundError(account_id)          # hide existence

    def operate(self, account_id: int) -> Account:
        account = self._load(account_id)
        if account.owner_id == self.user.id:
            return account
        if self.user.role == Role.ADMIN:
            raise ForbiddenError("Admins cannot move money on customer accounts")
        raise AccountNotFoundError(account_id)

# repository: filter in SQL
def get_all(self, owner_id=None):
    q = self.db.query(Account)
    if owner_id is not None:
        q = q.filter(Account.owner_id == owner_id)
    return q.all()
```



Common mistakes

- Filtering the list in Python after loading ALL accounts
- Checking the TO account in transfer (payee can be anyone)
- Returning 403 to customers: confirms the id exists
- Forgetting routes without account_id in the path (payments by id)
- Old bank.db without the owner_id column



Verify

```
pytest tests/test_auth5_ownership.py -v
-> 9 passed
```

TASK 5 · WHERE EACH CHECK GOES

Task 5: wiring the routes

```
# account_router.py
# routes with {account_id} in the path:
# GET /{account_id} -> can_view_account
# POST /{account_id}/deposit -> can_operate_account
# POST /{account_id}/withdraw -> can_operate_account
# GET /{account_id}/kyc-status -> can_view_account
@router.post("/{account_id}/deposit",
             response_model=AccountOut,
             dependencies=[Depends(can_operate_account)])

@router.post("/")
def create_account(payload: AccountCreate,
                  service: AccountService = Depends(),
                  user: User = Depends(get_current_user)):
    return service.create_account(payload.owner_name,
                                  owner_id=user.id)

@router.post("/transfer", response_model=TransferResult)
def transfer_money(..., access: AccountAccess = Depends()):
    access.operate(request.from_account_id) # FROM = mine
    scoped_key = (f"user:{access.user.id}:{idempotency_key}"
                  if idempotency_key else None)
    ... # use scoped_key where idempotency_key was used
```

```
# transaction_router.py
@router.get("/{account_id}/transactions", ...,
            dependencies=[Depends(can_view_account)])

# scheduled_payment_router.py
# POST /accounts/{account_id}/scheduled-payments
# -> can_operate_account
# GET /accounts/{account_id}/scheduled-payments
# -> can_view_account

@router.get("/scheduled-payments/{payment_id}", ...)
def get_scheduled_payment(
    payment_id: int,
    service: ScheduledPaymentService = Depends(),
    access: AccountAccess = Depends()):
    payment = service.get(payment_id)
    access.view(payment.account_id) # check PARENT
    return payment

# PATCH and DELETE: same, with access.operate(...)
```

Task 6: Roles (RBAC)



Goal

Customers cannot certify KYC or use /admin. Admins see all accounts, can certify KYC and disable users - but cannot move customer money.

Files

```
app/core/auth.py
app/routers/account_router.py
app/routers/admin_router.py
```

1 NUDGE

You need a reusable check "caller has one of these roles" that you can attach to any route or router. It needs the current user... which you already have as a dependency. Can a dependency take parameters?

2 POINTER

A dependency FACTORY: a function that takes the allowed roles and RETURNS the real dependency function. Use it as Depends(require_role(Role.ADMIN)) on the KYC PUT route and in APIRouter(dependencies=[...]) for the admin router.

3 ALMOST THERE

Inside checker: if user.role not in allowed_roles: raise ForbiddenError("Insufficient role for this operation", {"required_roles": list(allowed_roles), "your_role": user.role}); return user. In list_accounts: owner_id=None if user.role == Role.ADMIN else user.id.

Task 6: reference code

```
# auth.py
def require_role(*allowed_roles: str):
    def checker(user: User = Depends(get_current_user)) -> User:
        if user.role not in allowed_roles:
            raise ForbiddenError("Insufficient role for this operation",
                                {"required_roles": list(allowed_roles),
                                 "your_role": user.role})

        return user
    return checker

# account_router.py
@router.put("/{account_id}/kyc-status", response_model=KYCStatusOut,
            dependencies=[Depends(require_role(Role.ADMIN))])
def set_kyc_status(...): ... # body unchanged

@router.get("/", response_model=List[AccountOut])
def list_accounts(service: AccountService = Depends(),
                  user: User = Depends(get_current_user)):
    return service.list_accounts(
        owner_id=None if user.role == Role.ADMIN else user.id)

# admin_router.py
router = APIRouter(prefix="/admin", tags=["Admin"],
                  dependencies=[Depends(require_role(Role.ADMIN))])
```



Common mistakes

- Depends(require_role) without calling it - FastAPI then asks for *allowed_roles as query params
- Comparing to "admin" (lower-case) - roles are "ADMIN"
- Adding role checks inside AccountAccess.view() and blocking admins from reading



Verify

```
pytest tests/test_auth6_roles.py -v
-> 7 passed
pytest -v -> 108 passed
```

WHICH FAILING TEST POINTS TO WHICH BUG

Test failure decoder

Failing test	Most likely cause
<code>test_password_is_stored_hashed_not_plain</code>	<code>hash_password</code> returns the input, or register stores password instead of the hash
<code>test_cannot_self_register_as_admin (gets 201)</code>	<code>UserCreate</code> missing <code>extra="forbid"</code>
<code>test_token_claims_are_minimal_and_correct</code>	<code>sub</code> is an int, <code>iss</code> missing, or extra claims (e.g. username) added
<code>test_wrong_password_and_unknown_user_look_identical</code>	Two different error messages, or 500 from <code>user.hashed_password</code> when user is None
<code>test_alg_none_token_is_401 / test_wrong_issuer_is_401</code>	algorithms not pinned / <code>issuer=</code> not passed to <code>jwt.decode</code>
<code>test_route_requires_token[get-/accounts/]</code>	Legacy path open: dependency added to the v1 include instead of the account router
<code>test_cannot_transfer_from_someone_elses_account</code>	<code>transfer_money</code> never calls <code>access.operate(from_account_id)</code>
<code>test_idempotency_keys_are_scoped_per_user</code>	Raw Idempotency-Key used as the DB key; prefix it with the user id
<code>test_admin_can_view_but_not_move_customer_money</code>	<code>operate()</code> lets admins through, or <code>view()</code> blocks them
<code>test_deactivated_user_is_locked_out_immediately</code>	<code>get_current_user</code> does not check <code>user.is_active</code>
<code>test_not_found_v1_and_legacy (Extension 5)</code>	GET <code>/accounts/{id}</code> has no <code>can_view_account</code> dependency yet

MANUAL VERIFICATION

Checking by hand: Swagger and curl

Swagger UI

- 1 Open <http://127.0.0.1:8000/docs>
- 2 POST `/api/v1/auth/register` - create a user (password $\geq$ 15 chars)
- 3 Click Authorize (padlock, top right). Enter username + password, leave `client_id/secret` empty
- 4 Swagger calls `/auth/token` and adds "Authorization: Bearer ..." to every call
- 5 Try GET `/api/v1/auth/me`, then an account route. Logout and retry: 401

```
curl (Git Bash / macOS / Linux)
API=localhost:8000/api/v1

# log in -> token
curl -s -X POST $API/auth/token \
  -d "username=asha&password=Correct-Horse-Battery-9"

TOKEN=<paste access_token>

# who am I?
curl -s $API/auth/me -H "Authorization: Bearer $TOKEN"

# read the payload (no key needed!)
echo $TOKEN | cut -d. -f2 | base64 -d; echo

# no token -> 401 + WWW-Authenticate
curl -si $API/accounts/ | head -5
```

OPTIONAL

Stretch goals: hints only



S1 Refresh tokens

- `secrets.token_urlsafe(32)` as the refresh token
- Store `sha256(token)` in `refresh_tokens`, never the token
- On `/auth/refresh`: mark old one revoked, issue a new pair
- Old token reused? Revoke every token of that user (theft)



S2 Login throttling

- Key = username + client IP (`request.client.host`)
- Window of 15 min, max 5 failures
- Raise `HTTPException(429, headers={"Retry-After": "900"})`
- Reset the counter after a successful login



S3 Audit trail

- `performed_by` column on `Transaction`
- Pass the user into the service (or `AccountAccess.user`)
- `audit_events`: who, what, when, `request_id`
- Never log passwords or full tokens



S4 Revoke by jti

- Add `"jti": uuid4().hex` when creating tokens
- `POST /auth/logout` stores `jti` until exp
- `get_current_user`: denylisted `jti` -> 401
- Clean up rows after exp

Finish line

108

passed · 51 from Extension 5 + 57 new

Before you call it done, explain to your pair:

- Why the token payload is not a safe place for data
- Which line stops an alg:none token
- Why a disabled user is locked out before their token expires
- Why Ravi gets 404, not 403, on Asha's account
- Why an admin gets 403 on withdraw

Reference solution: [banking_api_ext6_solution.zip](#) (instructor)